# 14.1 通道注册表（Channel Registry）

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~280,000 tokens，输出 ~20,000 tokens（本章合计）

OpenClaw 的核心价值之一是**统一收件箱**——无论消息来自 WhatsApp、Telegram、Discord 还是 Slack，Gateway 都能以统一的方式处理。要实现这一点，首先需要一个**通道注册表**（Channel Registry）来管理所有已知的消息通道。本节分析注册表的设计和通道能力声明机制。

## 14.1.1 通道注册机制

`src/channels/registry.ts` 定义了 OpenClaw 的核心通道列表和元数据。

### 核心通道列表

```
// src/channels/registry.ts
export const CHAT_CHANNEL_ORDER = [
  "telegram",
  "whatsapp",
  "discord",
  "googlechat",
  "slack",
  "signal",
  "imessage",
] as const;

export type ChatChannelId = (typeof CHAT_CHANNEL_ORDER)[number];
export const DEFAULT_CHAT_CHANNEL: ChatChannelId = "whatsapp";
```

`CHAT_CHANNEL_ORDER` 不仅定义了所有核心通道的 ID，还通过数组顺序决定了它们在 UI 中的显示优先级——Telegram 排在最前面，因为它是"最简单的入门方式"。

每个通道都附带一份元数据（`ChannelMeta`），描述其显示名称、文档路径和简介：

```
// src/channels/registry.ts
const CHAT_CHANNEL_META: Record<ChatChannelId, ChannelMeta> = {
  telegram: {
    id: "telegram",
    label: "Telegram",
    selectionLabel: "Telegram (Bot API)",
    detailLabel: "Telegram Bot",
    docsPath: "/channels/telegram",
    blurb: "simplest way to get started — register a bot with @BotFather and get going.
    systemImage: "paperplane",
  },
  whatsapp: {
    id: "whatsapp",
    label: "WhatsApp",
    selectionLabel: "WhatsApp (QR link)",
    blurb: "works with your own number; recommend a separate phone + eSIM.",
    systemImage: "message",
  },
  // discord, slack, signal, googlechat, imessage...
};
```

## 通道别名

为了用户输入的便利性，注册表支持通道别名：

```
// src/channels/registry.ts
export const CHAT_CHANNEL_ALIASES: Record<string, ChatChannelId> = {
  imsg: "imessage",
  "google-chat": "googlechat",
  gchat: "googlechat",
};
```

这样用户在配置或命令行中输入 `imsg` 、 `google-chat` 或 `gchat` 都能被正确识别。 `normalizeChannelId()` 函数统一处理这种映射：

```
// src/channels/registry.ts
export function normalizeChannelId(raw?: string | null): ChatChannelId | null {
  const normalized = raw?.trim().toLowerCase();
  if (!normalized) return null;
  const resolved = CHAT_CHANNEL_ALIASES[normalized] ?? normalized;
  return CHAT_CHANNEL_ORDER.includes(resolved) ? resolved : null;
}
```

对于通过插件系统注册的外部通道（如 Microsoft Teams、Matrix 等），`normalizeAnyChannelId()` 会进一步查询插件注册表：

```
// src/channels/registry.ts
export function normalizeAnyChannelId(raw?: string | null): ChannelId | null {
  const key = raw?.trim().toLowerCase();
  if (!key) return null;
  const registry = requireActivePluginRegistry();
  const hit = registry.channels.find((entry) => {
    const id = String(entry.plugin.id).trim().toLowerCase();
    if (id === key) return true;
    // 也检查插件声明的别名
    return (entry.plugin.meta.aliases ?? []).some(
      (alias) => alias.trim().toLowerCase() === key
    );
  });
  return hit?.plugin.id ?? null;
}
```

## 14.1.2 通道能力声明

每个通道的功能不尽相同——Telegram 支持原生斜杠命令，WhatsApp 支持投票和反应表情，Discord 支持线程——OpenClaw 通过**能力声明**（Channel Capabilities）让 Gateway 知道每个通道能做什么。

能力声明定义在通道的 Dock 层（详见 6.1.3）中：

```
// 以 Telegram 为例
capabilities: {
  chatTypes: ["direct", "group", "channel", "thread"],
  nativeCommands: true,      // 支持 Bot API 原生斜杠命令
  blockStreaming: true,       // 支持块流（分段流式发送）
},

// 以 WhatsApp 为例
capabilities: {
  chatTypes: ["direct", "group"],
  polls: true,               // 支持投票
  reactions: true,            // 支持表情反应
  media: true,                // 支持多媒体消息
},

// 以 Discord 为例
capabilities: {
  chatTypes: ["direct", "channel", "thread"],
  polls: true,
  reactions: true,
  media: true,
  nativeCommands: true,       // 支持 Slash Commands
  threads: true,              // 支持线程/帖子
},
```

| 能力 | 说明 | 支持的通道 |
| --- | --- | --- |
| `chatTypes` | 支持的聊天类型 | 所有 |
| `nativeCommands` | 原生斜杠命令 | Telegram, Discord, Slack |
| `blockStreaming` | 块流式回复 | Telegram, Google Chat |
| `polls` | 投票功能 | WhatsApp, Discord |
| `reactions` | 表情反应 | WhatsApp, Discord, Slack, Signal, iMessage, Google Chat |
| `media` | 多媒体消息 | WhatsApp, Discord, Slack, Signal, iMessage, Google Chat |
| `threads` | 线程/帖子 | Discord, Slack, Google Chat |

Gateway 在处理消息时会检查目标通道的能力。例如，如果通道不支持 `reactions` ，就不会尝试发送确认反应；如果不支持 `threads` ，就不会在回复中包含线程相关的上下文。

## 14.1.3 通道 Dock 层

**Dock**（停靠层）是 OpenClaw 对通道行为的轻量级抽象。它在注册表和具体通道实现之间提供了一个统一的适配接口，使得共享代码（如消息分发、路由、命令处理）可以不依赖具体通道的重量级实现。

`src/channels/dock.ts` 中的 `ChannelDock` 类型定义了 Dock 的完整接口：

```
// src/channels/dock.ts
export type ChannelDock = {
  id: ChannelId;
  capabilities: ChannelCapabilities;        // 能力声明
  commands?: ChannelCommandAdapter;          // 命令适配
  outbound?: {
    textChunkLimit?: number;                 // 出站消息分块大小限制
  };
  streaming?: ChannelDockStreaming;          // 流式发送配置
  elevated?: ChannelElevatedAdapter;         // 提权适配
  config?: {
    resolveAllowFrom?: (...) => string[];    // 白名单解析
    formatAllowFrom?: (...) => string[];     // 白名单格式化
  };
  groups?: ChannelGroupAdapter;              // 群组行为适配
  mentions?: ChannelMentionAdapter;          // @提及适配
  threading?: ChannelThreadingAdapter;       // 线程适配
  agentPrompt?: ChannelAgentPromptAdapter;   // Agent 提示词适配
};
```

每个核心通道都在 `DOCKS` 对象中注册了自己的 Dock。以 WhatsApp 为例：

```
// src/channels/dock.ts
whatsapp: {
  id: "whatsapp",
  capabilities: { chatTypes: ["direct", "group"], polls: true, reactions: true, media:
  commands: {
    enforceOwnerForCommands: true,   // 只有号码拥有者可以执行命令
    skipWhenConfigEmpty: true,
  },
  outbound: { textChunkLimit: 4000 },
  config: {
    resolveAllowFrom: ({ cfg, accountId }) =>
      resolveWhatsAppAccount({ cfg, accountId }).allowFrom ?? [],
    formatAllowFrom: ({ allowFrom }) =>
      allowFrom.map(entry => normalizeWhatsAppTarget(entry)).filter(Boolean),
  },
  groups: {
    resolveRequireMention: resolveWhatsAppGroupRequireMention,
    resolveToolPolicy: resolveWhatsAppGroupToolPolicy,
  },
  mentions: {
    stripPatterns: ({ ctx }) => {
      const selfE164 = (ctx.To ?? "").replace(/^whatsapp:/, "");
      return selfE164 ? [escapeRegExp(selfE164), `@${escapeRegExp(selfE164)}`] : [];
    },
  },
},
```

`textChunkLimit` 的值因通道而异：

| 通道 | 分块上限 |
| --- | --- |
| Discord | 2000 字符 |
| Telegram / WhatsApp / Slack / Signal / iMessage / Google Chat | 4000 字符 |

对于通过插件系统注册的外部通道， `buildDockFromPlugin()` 函数会自动从插件的属性中构建 Dock：

```
// src/channels/dock.ts
function buildDockFromPlugin(plugin: ChannelPlugin): ChannelDock {
  return {
    id: plugin.id,
    capabilities: plugin.capabilities,
    commands: plugin.commands,
    outbound: plugin.outbound?.textChunkLimit
      ? { textChunkLimit: plugin.outbound.textChunkLimit }
      : undefined,
    groups: plugin.groups,
    mentions: plugin.mentions,
    threading: plugin.threading,
    // ...
  };
}
```

`listChannelDocks()` 函数将核心通道和插件通道合并，按优先级排序后返回：

```
// src/channels/dock.ts
export function listChannelDocks(): ChannelDock[] {
  const baseEntries = CHAT_CHANNEL_ORDER.map((id) => ({
    id, dock: DOCKS[id], order: getChatChannelMeta(id).order,
  }));
  const pluginEntries = listPluginDockEntries();
  const combined = [...baseEntries, ...pluginEntries];
  combined.sort((a, b) => {
    const orderA = a.order ?? (indexA === -1 ? 999 : indexA);
    const orderB = b.order ?? (indexB === -1 ? 999 : indexB);
    return orderA !== orderB ? orderA - orderB : String(a.id).localeCompare(String(b.id
  });
  return combined.map((entry) => entry.dock);
}
```

> **设计哲学**：Dock 层的注释中强调了"保持轻量"的原则——不引入监控、Web 登录、Puppeteer 等重量级依赖。这是因为 Dock 层会被 Gateway 的各个子系统广泛引用，如果它间接加载了具体通道的全部实现，就会导致不必要的内存开销和启动延迟。具体通道的重量级逻辑应该放在各自的插件模块中。

# 本节小结

OpenClaw 的通道注册表体系有以下核心特点：

1. **双层注册**：核心通道硬编码在 `CHAT_CHANNEL_ORDER` 中，外部通道通过插件注册表动态注册，两者最终合并为统一的列表
2. **能力声明**：每个通道显式声明自己支持的功能（聊天类型、反应、投票、线程等），Gateway 据此决定可用的交互方式
3. **轻量 Dock 层**：在注册表和具体实现之间提供统一的行为适配接口，保持共享代码路径不依赖重量级通道实现
4. **别名系统**：支持用户输入的多种缩写形式，降低配置门槛

下一节分析入站消息如何通过 Gateway 的处理流水线——从原始通道消息到统一的内部格式。